

Experiment - 2

Hosting static website using AWS S3

Aim - Hosting a website
Making publicly available

Theory

- static website = fixed files (HTML, CSS, JS)
- no dynamic change in content
eg) portfolios, documents.

S3

- simple storage ~~bucket~~ service
- storing, backing up files, hosting data
- stores inside buckets
- container / folders
- Scalable, secure, fast access
- every bucket has unique name
- easy deployment

Steps

1. log into AWS console with credentials
2. select region as Mumbai = ap-south-1
3. go to S3 bucket
4. click on create bucket
5. general purpose bucket
6. name it
7. ACL enabled
8. unblock all public IP
9. no need of bucket versioning
10. create bucket

1. go to tempdemo.com
2. download some templates
3. go to created bucket
4. upload files
6. click on objects
7. make public ACL

1. go to buckets
2. go to properties
3. go to static hosting → enable
4. write the ~~index.html~~
error.html

1. go to index.html
2. go to the url from there
3. ensure website is intact

How to copy & move files in S3 buckets.

Theory

- S3 explanation
- it is used to manage data
- mention allowed file types
- creating data back up
- easy deployment
- data migration
- data storage
- explain ← copy moving

Procedure

1. log into AWS console
2. change region to ap-south-1
3. Go to bucket
4. create a
 - general purpose
 - block IP public
 - ACL enabled
 - enable bucket versioning
5. go to objects
6. create 3 folders
 - Data 1
 - Data 2
 - Data 3
7. put 2 files in ~~image~~ Data 1
 - Image 1 } upload → wait
 - Image 2 }
8. Click on Image 1 → action → copy → choose D2 folder
9. click on Image 2 → action → move → choose D3 folder

Experiment - 4

IPS
How to launch a VM in AWS using PuTTY software

Theory

- how to create a EC2 instance in AWS
- connect to laptop using PuTTY
- EC2 = elastic cloud compute
 - ↳ service in AWS to create VM in cloud
 - ↳ has CPU, RAM, OS, storage, network access
- instead of buying comp. we rent one = EC2

PuTTY

- software to remotely connect to another com
- uses SSH
- like a remote control computer

establishes SSH connection (protocol)

↓
authenticate using pub key
↓
opens terminal access
↓
type linux commands (ls, pwd)

Procedure

1. Log into AWS console with creds
2. go to ec2 - south 1
3. EC2
4. select Amazon Machine Image = ubuntu
5. Instance type = t3.micro
6. key pair → new → RSA → download ppk
7. create instance

1. download PuTTY → finish wizard
2. Host = public IP v4
3. connection → ~~auth~~ SSH → Auth → Credential →
4. upload - ppk

1. security group
2. inbound rule — SSH — allow from all IP
- port 22

go to putty
log in as ubuntu
try & linux commands

Experiment - 5

Problem statement

Creating AWS resource using template code

Theory

- setting up resources = time consuming, error prone

- AWS : IaaS

- CloudFormation → module to create resource

- what to create

- how to create

- Config

- **VPC** Virtual private cloud
↳ control IP ranges
creating subnets

IG communication b/w internet & VPC

Stack = complete collection of AWS resources created from template

YAML file

AWS Template Format Version: '2010-09-09'

Description: create a VPC, two subnets & an EC2 instance

Resources

• MyVPC -
Type: AWS::EC2::VPC

Properties:

CidrBlock: 10.0.0.24/16

EnableDnsSupport: true

EnableHostDns: true

~~Properties:~~

Tags:

Key: Name

Value: MyVPC

create Internet Gateway

MyInternetGateway

Type: AWS::EC2::InternetGateway

Properties

VPCId: !Ref MyVPC

connect IG to VPC

AttachGateway

Type: AWS::EC2::VPCGatewayAttachment

Properties

CidrBlock: 10.0.0.24/16

InternetGatewayId: !Ref MyIntGateway

Make a public Route Table
Public Route Table

Type: AWS::EC2::RouteTable

Properties:
VPCId: !Ref myVPC

~~Attach~~ ~~to~~ ~~the~~ make public Route

Type: AWS::EC2::Route

Properties:

GatewayId: !Ref MyInternetGateway

RouteTableId: !Ref

Subnet 1

Type: AWS::EC2::Subnet

Properties:

VPCId:

CidrBlock

Subnet 2

Attach SI to Route Table

Type: AWS::EC2::SubnetRouteTableAssociation

Properties:

RouteTableId

SubnetId

EC2

Type: AWS::EC2::Instance

Properties:

InstanceType

KeyName

AMI

SecurityGroups

1. CloudFormation

2. Create Stack → Template → Upload yaml file

3. Give name

4. Create Stack

o/p - all green sections at end of bar

Experiment: Explain VPC components & its configurations.

Theory

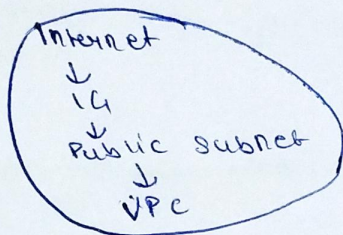
VPC - Virtual private cloud/network
- for defining ranges
- creating subnets
- controlling security

CUSTOMISED

Subnet - smaller division of VPC

IG - connects VPC & internet

Route table - determines which traffic goes where



Procedure

1. Log into AWS console using credentials
2. go to VPC → VPLs
3. create a VPC → give it a name
4. IPv4: 10.0.0.0 / 16
5. everything default

#subnet

1. in LHS pane → subnets
2. create one
3. attach to newly made VPC
4. IPv4 CIDR: 10.0.1.0 / 24
↳ 256 IPs

#IG

1. in LHS pane → internet gateway
2. give it a name
3. create
4. Actions → attach to VPC

#Route & Route Table

1. LHS pane → Route Table
2. link it to VPC
3. give it a name
4. create
5. Routes → new
↳ destination = 0.0.0.0 / 0
↳ IGW = connect to AWS

6. update

#connect subnet to RT

1. go to subnet association
2. edit
3. choose association

Experiment - 6

Automatic stop of EC2 instances using AWS Lambda & EventBridge

Theory

- Explain EC2

- AWS Lambda = computing that executes specific Java, Node, Py code on a schedule

- IAM roles

- EventBridge = cron job scheduler

- security

Procedure

1. Log into console AWS

2. go to EC2

↳ AMI
name
instance type
allow SSH/HTTP
security key pair

3. launch instance

IAM

4. go to IAM

5. Role → create role → AWS services →
Lambda → Full EC2 access

6. give role name

Lambda

1. go to AWS Lambda

2. create function → create from scratch

3. give function name

4. Python 3.14

5. give same IAM role

6. create fn.

7. scroll down to code & replace

```
import boto3
instance = ['i-...']
region = 'ap-south-1'
```

```
def lambda_handler(event, context):
```

```
    ec2 = boto3.client('ec2', region_name=region)
```

```
    ec2.stop_instances(InstanceIds=instance)
```

```
    print("EC2 successfully stopped")
```


A Event Budget

1. go to AWS Eventbridge
2. create rule
 - ↳ schedule cron expression
 - 30 11 3 * * 2 *
 - target: AWS service lambda function
 - Select right one
3. create rule

To my

~~EC2 instance → worker → terminal →~~

1. 2000 2000 2000 2000 2000 2000 2000 2000 2000 2000
 2. 2000 2000 2000 2000 2000 2000 2000 2000 2000 2000
 3. 2000 2000 2000 2000 2000 2000 2000 2000 2000 2000
 4. 2000 2000 2000 2000 2000 2000 2000 2000 2000 2000
 5. 2000 2000 2000 2000 2000 2000 2000 2000 2000 2000
 6. 2000 2000 2000 2000 2000 2000 2000 2000 2000 2000
 7. 2000 2000 2000 2000 2000 2000 2000 2000 2000 2000
 8. 2000 2000 2000 2000 2000 2000 2000 2000 2000 2000
 9. 2000 2000 2000 2000 2000 2000 2000 2000 2000 2000
 10. 2000 2000 2000 2000 2000 2000 2000 2000 2000 2000

[illegible]

creating RDS in AWS

Theory

- explain RDS
- cloud db service
- handles hardware, patching, security, scaling, back ups
- eg) MySQL, PostgreSQL
- appln: banking

Steps

1. log into acc
 2. go to Amazon RDS Australia
 3. create database
 - ↳ create easy
 - ↳ select engine
 - ↳ select name & done
 - idk } { master username (username)
 - { master password
 4. create db
 5. find endpoint name & port no
 .rds.amazonaws.com
 6. go to EC2 instance → security group
 7. find linked one (launch wizard)
 db
 8. go to Edit inbound rules
 - ↳ type = PostgreSQL
 - TCP
 - port 5432
 - source = My IP
- save rules

Experiment 8

PS Creating a cloudwatch & SNS alarm notification for services

Theory

- cloud monitoring important $\leftarrow$ Resources Billing
- CW = monitoring service
 - $\hookrightarrow$ collect metrics
 - $\hookrightarrow$ create an alarm
 - $\hookrightarrow$ sends notif
- eg CPU util, disk usage, network traffic
- SNS = simple notification service
 - Email, phone, push notif

Steps

1. create EC2 instance with same steps

2. # SNS

- $\hookrightarrow$ go to AWS SNS
- $\hookrightarrow$ create topic
 - type = standard
 - name = something

$\hookrightarrow$ create subscription

- $\hookrightarrow$ protocol = email
- endpoint = kulnikirazo @ —
& verity

3. # cloudwatch

- $\hookrightarrow$ metrics $\rightarrow$ EC2 $\rightarrow$ per instance metrics
- $\hookrightarrow$ CPU utilization
- $\hookrightarrow$ create alarm
 - Average
 - granite
 - set threshold $> 70\%$

$\hookrightarrow$ next

- $\hookrightarrow$ for notification - attach SNS
 - choose step 2 name

$\hookrightarrow$ add name & desc

4. # test

- $\hookrightarrow$ go to EC2 instance
- connect
- terminal in browser
- $>$ openssl speed